

Phương pháp bảo toàn trong tin học

He Lin

IOI 2004, đội tuyển tập huấn quốc gia Trung Quốc

Nguồn gốc: Conservation method in informatics

IOI China candidate team papers / 2004 / He Lin.pdf

Abstract

Bài viết đề xuất và tổng kết “phương pháp bảo toàn”, cùng một số ứng dụng của nó trong các kỳ thi tin học. Bản chất của bảo toàn là tìm đại lượng bất biến trong quá trình biến đổi. Phương pháp bảo toàn giúp ta nhảy qua, tránh được những chi tiết rối rắm, và nhìn thẳng vào bản chất của bài toán.

Từ khóa: phương pháp bảo toàn; bất biến.

1 Mở đầu

Đời sống thực tế và các bài toán thực tế đều rất phức tạp.

Bài toán 1. Hai quả cầu có khối lượng bằng nhau, vận tốc lần lượt là 5 m/s và 4 m/s, chuyển động ngược chiều nhau. Sau va chạm đàn hồi hoàn toàn, vận tốc của chúng là bao nhiêu?

Bài toán 2. 10g carbon và 10g O_2 phản ứng trong một bình kín trong một giờ. Khối lượng tổng cộng cuối cùng là bao nhiêu?

Bài toán 1 hẳn đã quen thuộc: bảo toàn động lượng và bảo toàn động năng cho ta hai phương trình, từ đó giải được vận tốc. Trên thực tế, quá trình va chạm của hai quả cầu rất phức tạp: hai cặp lực tác dụng, ảnh hưởng lẫn nhau và sinh công ra sao là điều không dễ phân tích. Nếu bỏ qua quá trình biến đổi cụ thể, ta rất dễ thấy hai đại lượng bất biến trong biến đổi là “động lượng” và “động năng”. Nắm được bất biến ấy, ta có thể bỏ qua chi tiết rườm rà và đi thẳng tới mục tiêu.

Bài toán 2 cũng tương tự. Phản ứng giữa C và O_2 cũng phức tạp. Ở các vùng cục bộ khác nhau, điều kiện khác nhau, có thể sinh ra CO, cũng có thể sinh ra CO_2 ; CO_2 và C còn có thể chuyển hóa ngược lại thành CO. Thực ra sẽ không ai liệt kê ba phương trình hóa học để phân tích. Trong một bình kín, dù biến đổi thế nào, tổng khối lượng chắc chắn không đổi, tức bảo toàn khối lượng. Nắm được điểm này, trong một giây ta có thể trả lời: 20g.

Hai ví dụ trên minh họa sinh động tác dụng của bảo toàn.

Đời sống thực tế và các bài toán thực tế phức tạp đến mức điều kiện và biến đổi quá nhiều; chỉ cần xét thiếu chặt chẽ một chút là có thể sai toàn bộ, hoặc do bản thân bài toán quá phức tạp mà không thể phân tích trực tiếp.

Nhưng nếu tìm được một, hai đại lượng bảo toàn, tức những bất biến trong biến đổi, bài toán sẽ được đơn giản hóa rất nhiều. Bỏ qua chi tiết, nắm mâu thuẫn chính: đó chính là phương pháp bảo toàn.

2 Một ví dụ đơn giản

Ví dụ 1

Cho một dãy số $a_1, a_2, a_3, \dots, a_n$. Mỗi lần ta được chọn tùy ý ba số kề nhau a_{i-1}, a_i, a_{i+1} và thực hiện thao tác sau, gọi là “thao tác trên a_i ”:

$$(a_{i-1}, a_i, a_{i+1}) \rightarrow (a_{i-1} + a_i, -a_i, a_i + a_{i+1}).$$

Cho dãy ban đầu và dãy mục tiêu. Hỏi có thể dùng các thao tác trên để biến dãy ban đầu thành dãy mục tiêu hay không?

Ví dụ, với dãy ban đầu

$$(1, 6, 9, 4, 2, 0)$$

và dãy mục tiêu

$$(7, -6, 19, 2, -6, 6),$$

có thể biến đổi như sau:

$$\begin{aligned}(1, 6, 9, 4, 2, 0) &\rightarrow (1, 6, 13, -4, 6, 0) \\ &\rightarrow (1, 6, 13, 2, -6, 6) \\ &\rightarrow (7, -6, 19, 2, -6, 6).\end{aligned}$$

Trong bản gốc, phần tử a_i được thao tác ở mỗi bước được in đậm.

Nếu dãy ban đầu là $(1, 2, 3)$, dãy mục tiêu là $(1, 3, 2)$, thì dù làm thế nào cũng không thể biến đổi từ dãy ban đầu sang dãy mục tiêu.

Input.txt	Output.txt
1 6 9 4 2 0	Yes
7 -6 19 2 -6 6	
Input.txt	Output.txt
1 2 3	No
1 3 2	

Ta phân tích bài toán này.

Thao tác có thứ tự trước sau. Chẳng hạn thao tác trên a_2 rồi thao tác trên a_3 , và thao tác trên a_3 rồi thao tác trên a_2 , có thể cho kết quả khác hẳn nhau.

Quan sát ví dụ:

$$\begin{aligned}(1, 6, 9, 4, 2, 0) &\rightarrow (1, 6, 13, -4, 6, 0) \\ &\rightarrow (1, 6, 13, 2, -6, 6) \\ &\rightarrow (7, -6, 19, 2, -6, 6).\end{aligned}$$

Các số nhìn có vẻ lộn xộn, không có quy luật. Xem kỹ quy tắc thao tác:

$$(a_{i-1}, a_i, a_{i+1}) \rightarrow (a_{i-1} + a_i, -a_i, a_i + a_{i+1}).$$

Trực quan mà nói, thao tác này giống như đem số ở giữa cộng sang hai bên rồi đổi dấu nó. Để thấy tổng trước và sau thao tác không đổi.

Ta có thể hào hứng đoán rằng: chỉ cần hai dãy có tổng bằng nhau thì chúng có thể biến đổi qua lại. Nhưng ví dụ thứ hai lập tức phủ định ý nghĩ này: $(1, 2, 3)$ và $(1, 3, 2)$ không thể đạt tới nhau. Từ $(1, 2, 3)$, thao tác duy nhất chỉ cho $(3, -2, 5)$; thao tác thêm lần nữa lại quay về $(1, 2, 3)$. Vì vậy nó không bao giờ biến thành $(1, 3, 2)$.

Tổng toàn cục chưa đủ, ta thử xét tổng cục bộ. Với thao tác trên ba phần tử:

(a_{i-1}, a_i, a_{i+1})	$(a_{i-1} + a_i, -a_i, a_i + a_{i+1})$
$S_1 = a_{i-1}$	$S'_1 = a_{i-1} + a_i$
$S_2 = a_{i-1} + a_i$	$S'_2 = a_{i-1}$
$S_3 = a_{i-1} + a_i + a_{i+1}$	$S'_3 = a_{i-1} + a_i + a_{i+1}$

Dễ thấy $S_3 = S'_3$, và $(S_1, S_2) = (S'_2, S'_1)$. Nói cách khác, chỉ cần đổi chỗ S_1 và S_2 trong (S_1, S_2, S_3) là thu được (S'_1, S'_2, S'_3) .

Mở rộng thêm một chút: đặt

$$S_i = a_1 + a_2 + \dots + a_i.$$

Thao tác trên (a_{i-1}, a_i, a_{i+1}) về bản chất tương đương với việc đổi chỗ S_{i-1} và S_i .

Chẳng hạn ví dụ đầu tiên:

$$\begin{aligned} (1, 6, 9, 4, 2, 0) &\rightarrow (1, 6, 13, -4, 6, 0) \\ &\rightarrow (1, 6, 13, 2, -6, 6) \\ &\rightarrow (7, -6, 19, 2, -6, 6). \end{aligned}$$

Sau khi chuyển sang dãy tổng tiền tố:

$$\begin{aligned} (1, 7, 16, 20, 22, 22) &\rightarrow (1, 7, 20, 16, 22, 22) \\ &\rightarrow (1, 7, 20, 22, 16, 22) \\ &\rightarrow (7, 1, 20, 22, 16, 22). \end{aligned}$$

Trong bản gốc, hai số được đổi chỗ ở mỗi bước được in đậm.

Với ví dụ thứ hai:

$$(1, 2, 3) \mapsto S = (1, 3, 6), \quad (1, 3, 2) \mapsto S = (1, 4, 6).$$

Thao tác trên $(1, 2, 3)$ thực chất chỉ là liên tục đổi chỗ 1 và 3 trong $S = (1, 3, 6)$. Dù làm thế nào cũng không thể biến thành $(1, 4, 6)$, vì 4 hoàn toàn không xuất hiện trong dãy tổng tiền tố ban đầu.

Ngoài ra còn một điểm nữa: những phần tử tham gia đổi chỗ chỉ là $S_1, \dots, S_{n-1}$; còn S_n bất động. Vì thế thuật toán là:

1. Kiểm tra S_n của hai dãy có bằng nhau không.
2. Kiểm tra hai đa tập $\{S_1, S_2, \dots, S_{n-1}\}$ có bằng nhau không.

Độ phức tạp thời gian là $\mathcal{O}(n \log n)$, do cần sắp xếp.

Một bài tưởng như rườm rà đã được giải rất nhẹ nhàng.

Như đã nói ở trên, quá trình biến đổi của thao tác rất phức tạp, gần như không có quy luật. Nếu bắt đầu bằng cách nghiên cứu đóng góp của từng thao tác lên tổng thể, ta sẽ rất dễ bế tắc.

Nhưng ta đã thực hiện một biến đổi nhỏ trên dãy ban đầu: lấy tổng tiền tố. Chính phép lấy tổng này làm bản chất của thao tác lộ ra. Thao tác từ dạng gây chóng mặt

$$(a_{i-1}, a_i, a_{i+1}) \rightarrow (a_{i-1} + a_i, -a_i, a_i + a_{i+1})$$

trở thành câu đơn giản: “đổi chỗ S_{i-1} và S_i ”.

Đây là một ví dụ kinh điển của việc ứng dụng bảo toàn. Đại lượng bảo toàn là đa tập $\{S_1, S_2, \dots, S_{n-1}\}$: nó luôn không đổi, không sinh thêm phần tử mới, cũng không có phần tử nào vô cớ biến mất; chỉ có thứ tự là thay đổi.

Bỏ qua chi tiết vụn vặt, ta đã nắm được bản chất.
Vì sao lại nghĩ tới việc lấy tổng? Bởi cấu trúc của

$$(a_{i-1}, a_i, a_{i+1}) \rightarrow (a_{i-1} + a_i, -a_i, a_i + a_{i+1})$$

rất dễ làm ta nhận ra tổng toàn cục được bảo toàn, từ đó liên tưởng thêm tới tổng cục bộ.

Cấu trúc của chính bài toán là căn cứ chủ yếu để chọn đại lượng bảo toàn. Liên tưởng và quy giản là chìa khóa để xây dựng đại lượng bảo toàn.

Bài này có đặc trưng khá rõ. Ta xét tiếp một bài khó hơn để thấy thêm sức mạnh của phương pháp bảo toàn.

3 Một ví dụ kín đáo

Ví dụ 2: Jumps

Đây là bài *Jump*, POI IV, vòng I, bài 2.

Trên một dãy ô vô hạn, một số ô có đặt quân cờ. Nếu trong một ô có quân, ta có thể lấy đi một quân ở ô ấy và đặt vào hai ô bên trái nó, mỗi ô một quân. Ngược lại, nếu hai ô liên tiếp đều có quân, ta có thể lấy đi mỗi ô một quân và đặt một quân vào ô bên phải của chúng. Hai phép này là nghịch đảo của nhau: một quân ở ô $p + 2$ tương đương với một quân ở ô p cộng một quân ở ô $p + 1$.

Nhiệm vụ hiện tại là: cho trạng thái ban đầu, hãy dùng các thao tác trên để đưa bàn cờ về trạng thái thỏa:

1. Mỗi ô có nhiều nhất một quân.
2. Không có hai ô kề nhau cùng có quân.

Nói ngắn gọn, trạng thái cuối cùng là trạng thái không thể thao tác tiếp.

Một ví dụ trong bản gốc:

Input.txt	Output.txt
001002001	000010101000

Ta phân tích bài toán này.

Đây lại là một bài toán thao tác. Yêu cầu không được có ô nào chứa hơn một quân. Trước hết xét nếu một ô cô lập có 3 quân thì làm thế nào. Bằng cách áp dụng các thao tác cơ bản, có thể biến ba quân chồng trên một ô thành một cấu hình có tổng số quân giảm đi một, đồng thời hai quân mới sinh ra cách nhau ba ô. Vì vậy ta có thể liên tục áp dụng các quy tắc sau:

1. Nếu hai ô kề nhau đều có quân, dùng thao tác cơ bản thứ hai: lấy chúng đi và đặt một quân vào ô bên phải.
2. Nếu một ô có ít nhất 3 quân, áp dụng phép biến đổi nói trên để làm số quân giảm đi.

Làm như vậy cho đến khi không thể làm tiếp theo hai quy tắc ấy. Khi đó các ô có quân chắc chắn là những ô không kề nhau, và mỗi ô chứa 1 hoặc 2 quân. Nếu còn ô chứa 2 quân, ta có thể biến hai quân ấy thành ba quân trải sang trái rồi tiếp tục xử lý. Mỗi lần chọn ô chứa 2 quân ở bên phải nhất để thao tác.

Nếu thao tác này tạo ra một cặp ô kề nhau cùng có quân, hoặc một ô có ít nhất 3 quân, ta có thể dùng quy tắc trên để làm tổng số quân giảm ít nhất 1. Xét trường hợp nó không tạo ra hai loại cấu hình ấy. Nếu ô mới nhận một quân vốn trống, ta đã giảm được số ô chứa 2 quân. Nếu không, ô ấy vốn có 1 quân; thêm một quân vào thì nó trở thành ô chứa 2 quân.

Như vậy ta đã dịch ô chứa 2 quân ở bên phải nhất sang trái một chút. Với ô chứa 2 quân mới sinh, tiếp tục “dịch chuyển” như vậy. Vì không thể có vô hạn ô chứa 2 quân, khi dịch tới một vị trí đủ xa bên trái chắc chắn sẽ kết thúc. Cuối cùng ta giảm được số ô chứa 2 quân.

Tổng hợp lại: ta hoặc làm tổng số quân giảm một, hoặc làm số ô chứa 2 quân giảm một. Vì số quân hữu hạn và số ô chứa 2 quân hữu hạn, luôn có thể biến bàn cờ về trạng thái thỏa yêu cầu cuối cùng.

Lập luận trên chứng minh sự tồn tại của nghiệm, đồng thời cũng cho một phương pháp khả thi: liên tục dùng hai quy tắc xử lý ô chứa 3 quân và ô chứa 2 quân để tìm đáp án cuối. Thực tế cho thấy đây cũng là một cách không tệ: chương trình chạy nhanh, độ phức tạp lập trình không quá cao.

Nhưng nghiệm có duy nhất không? Có phải chỉ có một nghiệm không? Ngoài ra, phương pháp này có phần ngẫu nhiên; độ phức tạp thời gian khó ước lượng và khó bảo đảm. Lời giải cũng đem lại cảm giác khá rời rạc, thúc đẩy ta tìm một cách tốt hơn.

Dễ nhận thấy hai thao tác đề bài cho là nghịch đảo của nhau. Quan sát

$$\text{một quân ở ô 4} + \text{một quân ở ô 5} \iff \text{một quân ở ô 6,}$$

ta có cảm giác rằng một quân ở ô thứ 4 cộng một quân ở ô thứ 5 tương đương với một quân ở ô thứ 6. Gọi là “tương đương” vì hai vế có thể chuyển hóa qua lại: thao tác thứ nhất thực hiện $6 \rightarrow 4 + 5$, thao tác thứ hai thực hiện $4 + 5 \rightarrow 6$.

Từ đó nảy sinh một ý tưởng thú vị: tại sao không gán cho mỗi ô một giá trị? Giả sử giá trị của ô thứ i là F_i . Khi đó ta cần

$$F_{i+1} = F_i + F_{i-1}.$$

Đây chẳng phải là dãy Fibonacci sao?

Nếu thật sự làm được vậy, trong mọi biến đổi, tổng giá trị của tất cả quân cờ sẽ không đổi. Thực tế đúng là như vậy. Xét ví dụ ở trên. Gán các giá trị Fibonacci

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots$$

cho các ô liên tiếp. Trạng thái ban đầu có tổng giá trị

$$2 \cdot 1 + 8 \cdot 2 + 34 \cdot 1 = 52.$$

Trạng thái cuối có tổng giá trị

$$5 + 13 + 34 = 52.$$

Hai tổng giá trị bằng nhau.

Bây giờ vấn đề là dùng điều này như thế nào. Nếu trạng thái ban đầu có tổng giá trị là 52, làm sao biết trong trạng thái mục tiêu, giá trị 52 sẽ được phân bố ra sao?

Vì mỗi ô có nhiều nhất một quân và không tồn tại hai ô kề nhau cùng có quân, kết quả cuối cùng chắc chắn là tổng của một số số Fibonacci không kề nhau. Gọi cách biểu diễn một số thành tổng các số Fibonacci không kề nhau là **phân tích Fibonacci**.

Định lý. Mọi số tự nhiên n có phân tích Fibonacci duy nhất, với $F_1 = 1, F_2 = 2$.

Chứng minh. Gọi F_p là số Fibonacci lớn nhất không vượt quá n . Khi đó F_p chắc chắn phải xuất hiện trong phân tích. Nếu không, tổng lớn nhất có thể tạo từ các số Fibonacci không kề nhau và nhỏ hơn F_p vẫn nhỏ hơn F_p .

Khi p chẵn:

$$\begin{aligned} F_{p-1} + F_{p-3} + \cdots + F_5 + F_3 + F_1 &= F_{p-1} + F_{p-3} + \cdots + F_5 + F_3 + F_2 - 1 \\ &= F_{p-1} + F_{p-3} + \cdots + F_5 + F_4 - 1 \\ &= \cdots \\ &= F_{p-1} < F_p \leq n. \end{aligned}$$

Khi p lẻ:

$$\begin{aligned} F_{p-1} + F_{p-3} + \cdots + F_6 + F_4 + F_2 &= F_{p-1} + F_{p-3} + \cdots + F_6 + F_4 + F_2 + F_1 - 1 \\ &= F_{p-1} + F_{p-3} + \cdots + F_6 + F_4 + F_3 - 1 \\ &= \cdots \\ &= F_{p-1} < F_p \leq n. \end{aligned}$$

Do đó F_p phải có trong phân tích. Tạm gọi giá trị hiện tại là N , đặt $n \leftarrow N - F_p$, rồi tìm F_q lớn nhất không vượt quá n . Chắc chắn $q \neq p - 1$; nếu không, khi cộng ngược lại với F_p ta có

$$F_p + F_q \leq N, \quad F_p + F_{p-1} = F_{p+1} \leq N,$$

mâu thuẫn với việc F_p là số Fibonacci lớn nhất không vượt quá N .

Lặp lại quá trình này, cuối cùng chắc chắn phân tích xong. Định lý được chứng minh.

Phương pháp phân tích là duy nhất; đồng thời thuật toán mô phỏng ở trên đã chứng minh tính tồn tại của nghiệm. Vì vậy cách phân tích này chắc chắn là một nghiệm hợp lệ.

Khi lập trình cần dùng số nguyên lớn: trước hết tính tổng giá trị của dữ liệu vào, sau đó thực hiện phân tích Fibonacci và xuất nghiệm.

Như vậy ta đã chứng minh được cả sự tồn tại lẫn tính duy nhất của nghiệm, và đã có hai thuật toán khả thi: mô phỏng và chọn theo Fibonacci.

Thuật toán mô phỏng khó ước lượng độ phức tạp thời gian, và ý tưởng khá rời rạc; ưu điểm là lập trình không khó, tốc độ cũng khá nhanh. Thuật toán chọn theo Fibonacci khó hiểu hơn một chút, rất tinh tế và đem lại cảm giác đẹp về mặt toán học; nhưng cần dùng số nguyên lớn, nên độ khó lập trình cũng không thua mô phỏng nhiều.

Còn một vấn đề cuối: khi dùng Fibonacci để gán trọng số cho các ô, phải bắt đầu gán 1, 2, 3, 5, ... từ một ô nào đó. Bắt đầu từ đâu mới bảo đảm đáp án cuối cùng đúng? Nói cách khác, trong kết quả cuối, quân cờ bên trái nhất có thể lệch tới vị trí nào?

Về lý thuyết ta có thể bắt đầu gán từ vô cực bên trái, đủ để chứng minh tính duy nhất. Nhưng khi lập trình, phải xét vị trí cụ thể; bắt đầu càng về bên phải càng tốt.

Thực ra ta có thể tìm đáp án từ thuật toán mô phỏng. Theo cách xử lý ô chứa 3 quân và 2 quân, đặt M là tổng số quân:

1. Thao tác trên ô chứa 3 quân nhiều nhất đẩy quân sang trái $2 \log_3 M$ vị trí.
2. Khi chỉ còn các ô chứa 1 hoặc 2 quân, thao tác trên ô chứa 2 quân thường sinh ra ô chứa 3 quân mới, rồi quy về thao tác trên 3 quân.
3. Nếu thao tác trên ô chứa 2 quân không tạo ô chứa 3 quân mới hoặc cặp ô kề nhau cùng có quân, thì chỉ có thể liên tục thao tác trên ô chứa 2 quân bên phải nhất, làm nó dịch dần sang trái. Cuối cùng thao tác này chỉ đẩy quân bên trái nhất thêm 2 vị trí. Đóng góp của nó vào dịch trái là hằng số.

Nói cách khác, nếu trong dữ liệu vào quân cờ bên trái nhất ở vị trí a , thì bắt đầu đánh số từ khoảng $a - 2 \log_3 M$ là đủ. Để an toàn có thể lấy rộng hơn, chẳng hạn $4 \log_3 M$. Ở đây $M \leq 10^9$.

Lời giải của bài này làm người ta sáng mắt. Từ “mô phỏng quân cờ” đến dãy Fibonacci, hai thứ tưởng như chẳng liên quan lại có thể nối với nhau.

Điều đáng nói nhất là phương pháp Fibonacci hoàn toàn bỏ qua chi tiết cụ thể. Quân cờ di chuyển ra sao, dùng quy tắc thứ nhất hay thứ hai, đều không cần quan tâm. Điều ta cần là kết quả cuối cùng; và kết quả ấy thỏa một điều kiện: tổng giá trị được bảo toàn.

Nhờ bất biến trong biến đổi này, ta giải quyết bài toán rất nhẹ nhàng.

“Tổng giá trị” là khái niệm không có trong đề bài. Nói rộng hơn, “giá trị” cũng không xuất hiện trong đề. Dù đều là ô và quân cờ, quy tắc lại gán cho chúng những địa vị khác nhau. Chính trên cơ sở quan sát quy tắc và tổng kết đặc điểm, ta mới chuyển được tư duy sang khái niệm “giá trị”, rồi tiếp tục đi đúng vào dãy Fibonacci.

4 Bài toán leo cầu thang

Ví dụ 3: Conqueror’s Battalion

Đây là bài *Conqueror’s Battalion*, CEOI 2002, ngày 1, bài 1. Nội dung tóm tắt như sau.

Có tổng cộng n bậc thang; trên một số bậc có một số binh sĩ của bạn. Bạn phải chia tất cả binh sĩ thành hai nhóm, rồi kẻ địch sẽ cho biết nhóm nào được giữ lại và nhóm nào bị tiêu diệt. Những binh sĩ được giữ lại đi lên một bậc.

Sau đó bạn lại chia nhóm các binh sĩ còn lại, kẻ địch lại chọn một nhóm giữ lại; nhóm được giữ lại lại đi lên một bậc.

Cứ lặp lại như vậy. Nếu cuối cùng có một binh sĩ lên tới đỉnh, tức lên bậc thứ n , bạn thắng; nếu toàn bộ binh sĩ bị tiêu diệt, kẻ địch thắng.

Bây giờ nhập n và số binh sĩ ở từng bậc. Nhiệm vụ là phán đoán liệu bạn có thể thắng hay không; nếu có thể thắng, còn phải tương tác với thư viện, trong đó thư viện đóng vai kẻ địch, và bạn phải chia nhóm thích hợp để cuối cùng giành thắng lợi.

Ta phân tích bài toán.

Hãy đặt mình vào vị trí kẻ địch: nếu đối phương chia binh sĩ thành hai nhóm, bạn sẽ giết nhóm nào? Có phải nhóm đông người nhất không? Không.

Chẳng hạn ở bậc $n - 1$ có 1 người, người ấy tự thành một nhóm; ở bậc 1 có 100 người, họ là một nhóm. Bạn sẽ giết ai? Tất nhiên là người ở bậc $n - 1$. Nếu không giết, vòng sau người ấy sẽ lên bậc n , đối phương sẽ thắng.

Vì vậy, người đứng ở bậc $n - 1$ đe dọa hơn người đứng ở bậc 1; hoặc nói cách khác, trong ngắn hạn có thể gây nguy hại lớn hơn. Nói chung, đứng càng cao thì càng nguy hiểm. Điều này gợi ý rằng địa vị của binh sĩ là khác nhau; không thể lấy số lượng người làm tiêu chuẩn phán đoán duy nhất.

Quay lại phía ta, hãy xem làm thế nào để thắng.

Giả sử chỉ có 2 người đứng ở bậc i . Ta tuyệt đối không thể đặt họ vào cùng một nhóm; nếu không, vòng sau kẻ địch có thể làm cả hai biến mất. Vì vậy chắc chắn phải chia hai người vào hai nhóm khác nhau. Sau khi kẻ địch làm một người chết, người còn lại sẽ lên bậc $i + 1$.

Nói cách khác: hai người đứng ở bậc i có tác dụng tương đương một người đứng ở bậc $i + 1$.

Tương tự ví dụ 2, đặt W_i là giá trị của một người đứng ở bậc i . Khi đó

$$2W_i = W_{i+1}.$$

Đặt $W_0 = 1$, ta có $W_i = 2^i$.

Mục tiêu cuối cùng là để một người đứng trên bậc thứ n , tức tổng giá trị của trạng thái mục tiêu ít nhất là 2^n .

Giả sử tổng giá trị của trạng thái ban đầu đúng bằng 2^n . Khi chia nhóm, ta cố chia thành hai nhóm sao cho tổng giá trị mỗi nhóm đều là 2^{n-1} . Dù kẻ địch xóa nhóm nào, nhóm còn lại luôn có tổng giá trị 2^{n-1} . Sau đó binh sĩ còn lại leo lên một bậc, tương đương mỗi người nhân giá trị với 2, tổng giá trị lại thành 2^n .

Vì vậy trong quá trình chơi, tổng giá trị được bảo toàn: luôn giữ là 2^n . Do số người không ngừng giảm, khi cuối cùng chỉ còn một người, người ấy chắc chắn đã lên đỉnh.

Ngược lại, nếu tổng giá trị $< 2^n$, sau khi chia thành hai nhóm, chắc chắn có một nhóm có tổng giá trị $< 2^{n-1}$. Kẻ địch có thể giữ lại nhóm đó. Sau khi nhóm ấy leo lên một bậc, tổng giá trị vẫn $< 2^n$. Trong suốt quá trình biến đổi, tổng giá trị luôn $< 2^n$, tức dù làm thế nào cũng không thể lên đỉnh.

Nếu tổng giá trị $> 2^n$, ta luôn có thể chia thành hai nhóm sao cho cả hai đều có tổng giá trị $\geq 2^{n-1}$. Khi đó trong quá trình biến đổi, tổng giá trị luôn $\geq 2^n$. Số người không ngừng giảm, cuối cùng nhất định có một người lên đỉnh.

Cụ thể chia như thế nào có thể dùng tham lam từ lớn đến nhỏ; ở đây không nói chi tiết, vì không liên quan tới chủ đề của bài viết. Điều cần nhấn mạnh ở đây là phương pháp gán trọng số này.

Việc phán đoán có thể thắng hay không đã chuyển thành phán đoán tổng giá trị có $\geq 2^n$ hay không. Nếu có, ta có thể tìm cách chia nhóm để tổng giá trị luôn giữ trên 2^n , và khi số người giảm dần, cuối cùng thắng.

“Tổng giá trị luôn giữ trên 2^n ” chính là bảo toàn của bài này. Nói nghiêm ngặt, nó không còn là quan hệ “bằng” nữa, mà là một quan hệ “lớn hơn”. Có thể xem đây là bảo toàn theo nghĩa rộng.

Nhưng tư tưởng cơ bản vẫn là tìm bất biến trong biến đổi. Gán trọng số nhị phân cho các bậc thang là một thử nghiệm rất tốt, và thử nghiệm này cũng được xây dựng trên cơ sở quan sát và liên tưởng kỹ lưỡng. Kết hợp đặc điểm của bài toán để tìm bất biến trong biến đổi.

Tổng kết

Các bài toán trên đều có một mức độ ẩn giấu nhất định; ban đầu không thấy rõ mảnh khoe nào. Chỉ thông qua một phép biến đổi nhất định, chẳng hạn lấy tổng hoặc gán trọng số, ta mới phát hiện bản chất của chúng.

Vì sao ví dụ 1 lấy tổng? Vì sao ví dụ 2 dùng trọng số Fibonacci? Vì sao ví dụ 3 lại dùng nhị phân? Nhấn mạnh thêm lần nữa:

Cấu trúc của chính bài toán là căn cứ chủ yếu để chọn đại lượng bảo toàn. Liên tưởng và quy giản là chìa khóa để xây dựng đại lượng bảo toàn.

Quan sát và liên tưởng là quan trọng nhất. Quan sát là quan sát quy luật của dữ liệu, đặc điểm của quy tắc, v.v. Liên tưởng là dựa trên những đặc điểm đã quan sát được để nối với kho tri thức đã có, hoặc tự sáng tạo.

Tích lũy tri thức rất quan trọng. Nếu không, dù đã quy nạp đúng đặc điểm, ta cũng khó so sánh, sáng tạo một cách hiệu quả.

Ngoài ra, nếu muốn dùng “phương pháp bảo toàn” để giải bài, tư duy sáng tạo rất quan trọng. Có nhiều đại lượng bảo toàn không thể tìm thấy trực tiếp trong đề bài, mà phải tự tạo ra. Một ví dụ kinh điển là Fibonacci heap.

Bản thân cài đặt Fibonacci heap không dùng phương pháp bảo toàn, nhưng khi phân tích độ phức tạp lại dùng “phương pháp thế năng”. Phương pháp này xây dựng một hàm rồi chứng minh độ phức tạp khẩu

hao của thao tác xóa trong Fibonacci heap là $\mathcal{O}(\log n)$. Đây cũng là một ứng dụng rất hay của tư tưởng bảo toàn.

Thật ra “bảo toàn” có ở khắp nơi. Việc vô tình lập một phương trình cũng là bảo toàn. Bảo toàn được nói trong bài này là một phương pháp tư duy và một cách giải quyết bài toán giữ vai trò chủ đạo trong đề.

Có lẽ ai cũng đã nhiều lần dùng tư tưởng này, dù có ý thức hay không. Bài viết chỉ muốn quy nạp các đặc điểm chính của phương pháp ấy, nhấn mạnh nó, và gọi sự chú ý. Sau này khi gặp bài toán, nếu có ý thức liên tưởng theo hướng này, có thể sẽ thu được kết quả bất ngờ.

Tài liệu tham khảo

- Đề CEOI: ví dụ 3.
- Đề POI: ví dụ 2.
- Đề kiểm tra nội bộ Yali: ví dụ 1.

Lời cảm ơn

Tác giả cảm ơn Jin Kai, Trường Trung học Changjun, Trường Sa, vì sự giúp đỡ trong ví dụ 3.

Tác giả cảm ơn bạn Yao Jinyu, Trường Trung học Yali, Trường Sa, đã cung cấp ví dụ 1.

A Nguyên đề của ví dụ 2: Jumps

POI IV, vòng I, bài 2

Jumps là một trò chơi bàn cờ trên một dải ô vuông vô hạn. Bàn cờ không có giới hạn trái hay phải. Trên các ô có hữu hạn quân cờ; một ô có thể có nhiều hơn một quân. Ta giả sử ô không rỗng bên trái nhất được đánh số 0. Các ô bên phải được đánh số bằng các số tự nhiên liên tiếp 1, 2, 3, ..., còn các ô bên trái được đánh số âm $-1, -2, -3, \dots$

Cấu hình quân cờ trên bàn được mô tả như sau: với mỗi ô có ít nhất một quân, cho số hiệu ô và số quân đứng trên ô ấy. Có hai loại nước đi có thể thay đổi cấu hình: nhảy sang phải và nhảy sang trái.

Nhảy sang phải gồm việc lấy khỏi bàn hai quân: một quân từ ô p , một quân từ ô $p + 1$, rồi đặt một quân vào ô $p + 2$. Nhảy sang trái gồm việc lấy một quân từ ô $p + 2$, rồi đặt hai quân lên bàn: một quân ở ô $p + 1$, một quân ở ô p .

Một cấu hình được gọi là cuối cùng nếu mỗi cặp ô kề nhau chứa nhiều nhất một quân. Với mọi cấu hình, có đúng một cấu hình cuối cùng có thể đạt được sau một số hữu hạn nước đi.

Nhiệm vụ

Viết chương trình:

- đọc mô tả cấu hình ban đầu từ tệp văn bản SK0.IN;
- tìm cấu hình cuối cùng có thể đạt được từ cấu hình đã cho và ghi kết quả vào tệp văn bản SK0.OUT.

Dữ liệu vào

Dòng đầu của tệp SK0.IN chứa một số nguyên dương n , $1 \leq n \leq 10000$, là số ô không rỗng trong cấu hình ban đầu. Trong mỗi dòng trong n dòng tiếp theo là mô tả của một ô không rỗng trong cấu hình ban đầu. Mô tả gồm hai số nguyên: số thứ nhất là số hiệu ô, số thứ hai là số quân đứng trên ô đó. Các mô tả được cho theo thứ tự tăng dần của số hiệu ô. Số hiệu ô lớn nhất không vượt quá 10000, và số quân trên một ô không vượt quá 100000000.

Dữ liệu ra

Dòng đầu của tệp SK0.OUT phải chứa cấu hình cuối cùng có thể đạt được từ cấu hình đã cho. Cụ thể, dòng này chứa các số hiệu ô không rỗng, theo thứ tự tăng dần, của cấu hình cuối. Các số cách nhau bởi một dấu cách.

Ví dụ

Với tệp SK0.IN:

2
05
33

đáp án đúng trong tệp SK0.OUT là:

-4 -1 1 3 5

B Nguyên đề của ví dụ 3: Conquer

Đề bài

Có hai người chơi một trò chơi tên là *Conquer*. Ban đầu phe chinh phục có nhiều nhất 1 000 000 000 binh sĩ. Các binh sĩ đứng trên N cao địa ($2 \leq N \leq 2000$), mỗi cao địa có một số binh sĩ. Phe chinh phục có thể chia binh sĩ trên các cao địa thành hai đội theo cách tùy ý. Phe bị chinh phục có thể yêu cầu một trong hai đội rời đi; các binh sĩ còn lại lần lượt chiếm cao địa cao hơn một bậc. Nếu cuối cùng phe chinh phục có ít nhất một binh sĩ tới đỉnh cao nhất, phe chinh phục thắng; nếu không, trò chơi tiếp tục cho đến khi không còn binh sĩ, và phe chinh phục thua.

Đây là một bài toán tương tác. Bộ chấm cung cấp cho bạn một thư viện ban đầu. Giả sử bạn là phe chinh phục; bạn sẽ dựa vào thư viện này để chơi. Mỗi vòng, bạn cần cung cấp cho thư viện một phương án chia đội. Thư viện trả về 1 hoặc 2: 1 nghĩa là giữ lại đội bạn mô tả cho thư viện, 2 nghĩa là giữ lại phần binh sĩ còn lại. Nếu bạn thắng hoặc không còn binh sĩ để chơi, trò chơi kết thúc và thư viện tự động dừng chương trình; bạn không được dùng bất kỳ cách nào khác để tự kết thúc chương trình.

Giao diện thư viện

Thư viện libconq cung cấp hai thủ tục/hàm:

- `start`: trả về N và một mảng `stairs`, biểu diễn số binh sĩ trên từng cao địa; `stairs[i]` là số binh sĩ ở cao địa i , trong đó `stairs[1]` là đỉnh cao nhất.

- step: nhận mảng tham số vào subset gồm N phần tử, mô tả phương án chia đội một lần; hàm trả về 1 hoặc 2 theo ý nghĩa trên.

Nếu cung cấp một cách chia không hợp lệ, thư viện sẽ tự động dừng chương trình và bạn được 0 điểm.

Mô tả thủ tục trong FreePascal và C/C++:

```
procedure start(var N: longint; var stairs: array of longint);
function step(subset: array of longint): longint;
void start(int *N, int *stairs);
int step(int *subset);
```

Dưới đây là ví dụ sử dụng thư viện trong FreePascal và C/C++. Chương trình gồm hai phần: bắt đầu trò chơi, rồi chọn ngẫu nhiên một đội binh sĩ. Chương trình thực tế của bạn có thể chứa một vòng lặp qua từng lượt chơi. Có thể định nghĩa mảng như trong ví dụ. Lưu ý: khi trả về, FreePascal có thể dùng đúng cách định nghĩa, còn C/C++ chỉ dùng chỉ số từ 1 đến N .

Ví dụ FreePascal.

```
uses libconq;
var
    stairs: array[1..2000] of longint;
    subset: array[1..2000] of longint;
    i, N, result: longint;
...
start(N, stairs);
...
for i := 1 to N do
    if random(2) = 0 then
        subset[i] := 0
    else
        subset[i] := stairs[i];
result := step(subset);
...
```

Ví dụ C/C++.

```
#include "libconq.h"
int stairs[2001];
int subset[2001];
int i, N, result;
...
start(&N, stairs);
...
for (i = 1; i <= N; i++)
    if (rand() % 2 == 0)
        subset[i] = 0;
    else
        subset[i] = stairs[i];
result = step(subset);
...
```

Chương trình FreePascal phải dùng `uses libconq` để liên kết thư viện; chương trình C/C++ dùng `#include "libconq.h"`. Khi đó chương trình sẽ được biên dịch cùng các tham số trong `libconq.c`.

Ví dụ tương tác

You	Library
<code>start(N,stairs)</code>	<code>N=8, stairs=(0,1,1,0,3,3,4,0)</code>
<code>Step((0,1,0,0,1,0,1,0))</code>	returns 2
<code>Step((0,1,0,0,0,1,0,0))</code>	returns 2
<code>Step((0,0,0,3,2,0,0,0))</code>	returns 1
<code>Step((0,0,2,0,0,0,0,0))</code>	returns 2
<code>Step((0,1,0,0,0,0,0,0))</code>	returns 2
<code>Step((0,1,0,0,0,0,0,0))</code>	no return: you won

Tài nguyên

Bạn có thể tạo trước một tệp `libconq.dat`. Tệp này gồm hai dòng: dòng đầu là N , số cao địa; dòng tiếp theo chứa N số nguyên, lần lượt biểu diễn số binh sĩ trên cao địa $1..N$. Ban đầu cao địa 1 không có binh sĩ.

```
libconq.dat
8
01103340
```